AI ASSISTED CODING

ASSIGNMENT-12.3

NAME:M.AKHIL REDDY

ROLL NO:2303A52315

BATCH-45

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

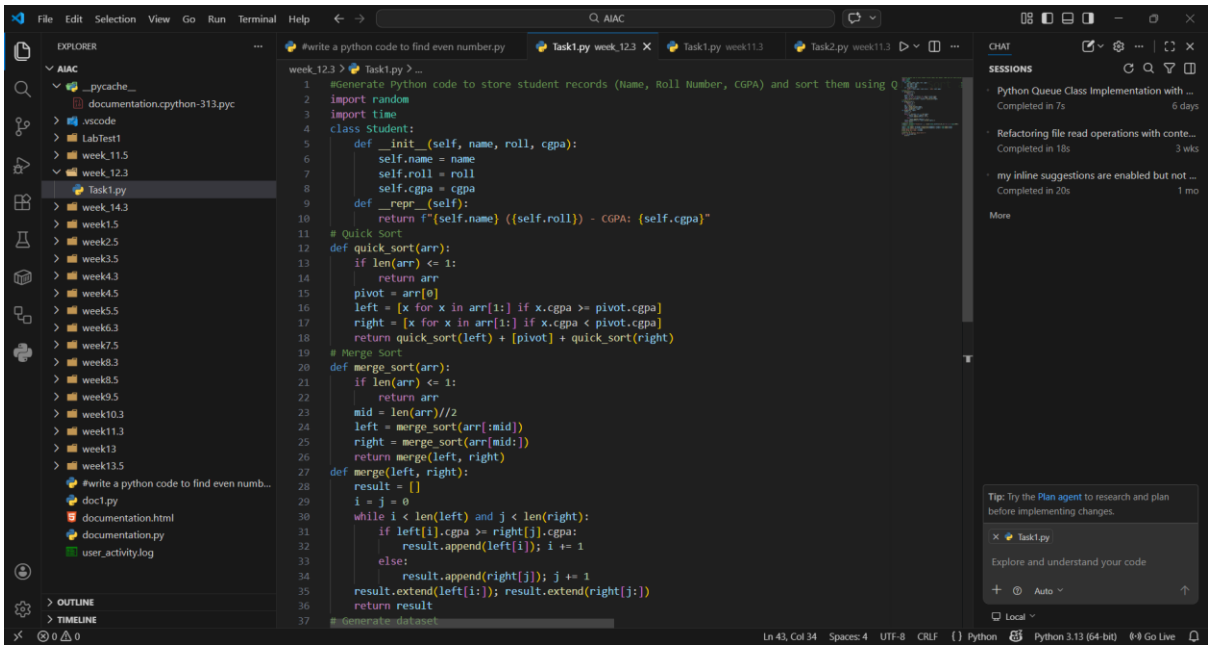
Expected Outcome

• Correctly sorted student records.

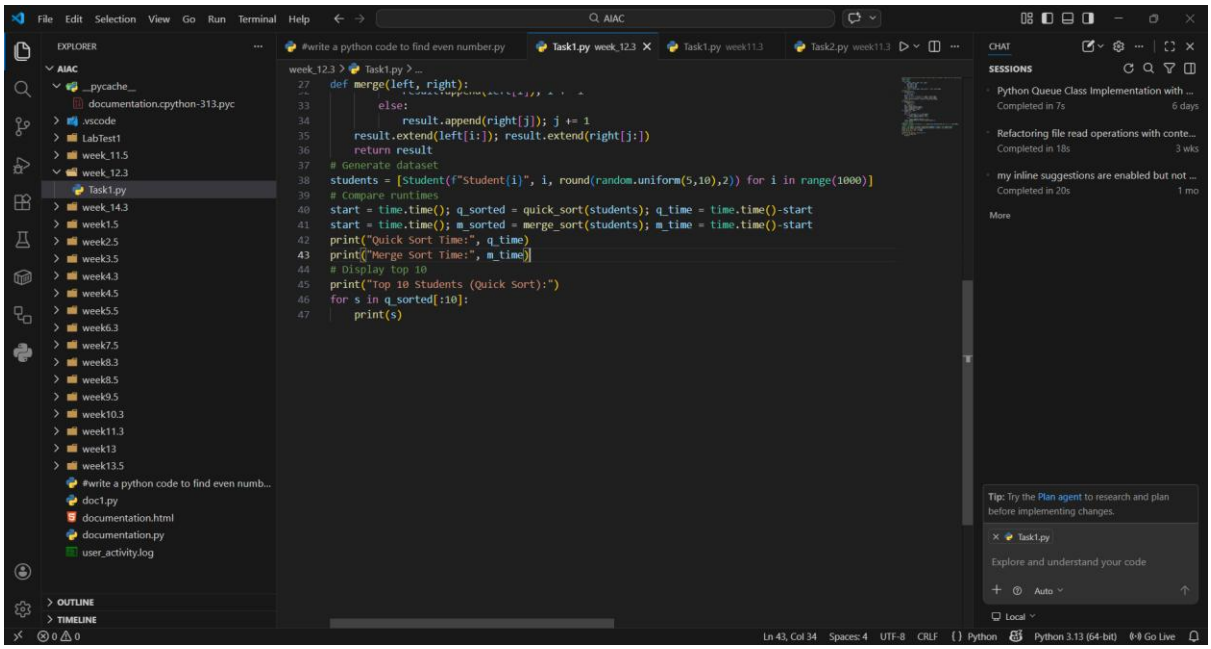
- **Performance comparison between Quick Sort and Merge Sort.**
- **Clear output of top-performing students.**

PROMPT: “Generate Python code to store student records (Name, Roll Number, CGPA) and sort them using Quick Sort and Merge Sort, then compare runtime performance and display the top 10 students."

CODE:



```python
#Generate Python code to store student records (Name, Roll Number, CGPA) and sort them using Q
import random
import time
class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
        self.cgpa = cgpa
    def __repr__(self):
        return f"{self.name} ({self.roll}) - CGPA: {self.cgpa}"
# Quick Sort
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x.cgpa >= pivot.cgpa]
    right = [x for x in arr[1:] if x.cgpa < pivot.cgpa]
    return quick_sort(left) + [pivot] + quick_sort(right)
# Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result
# Generate dataset
```



```python
def merge(left, right):
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result
# Generate dataset
students = [Student(f"Student{i}", i, round(random.uniform(5,10),2)) for i in range(1000)]
# Compare runtimes
start = time.time(); q_sorted = quick_sort(students); q_time = time.time()-start
start = time.time(); m_sorted = merge_sort(students); m_time = time.time()-start
print("Quick Sort Time:", q_time)
print("Merge Sort Time:", m_time)
# Display top 10
print("Top 10 Students (Quick Sort):")
for s in q_sorted[:10]:
    print(s)
```

OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\
hon.exe c:/Users/krish/OneDrive/Desktop/AIAC/week_12.3/Task1.py
Quick Sort Time: 0.004896879196166992
Merge Sort Time: 0.002748727798461914
Top 10 Students (Quick Sort):
Student548 (548) - CGPA: 10.0
Student332 (332) - CGPA: 10.0
Student727 (727) - CGPA: 9.99
Student390 (390) - CGPA: 9.99
Student953 (953) - CGPA: 9.98
Student385 (385) - CGPA: 9.98
Student682 (682) - CGPA: 9.97
Student107 (107) - CGPA: 9.96
Student148 (148) - CGPA: 9.95
Student52 (52) - CGPA: 9.95
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

EXPLANATION:

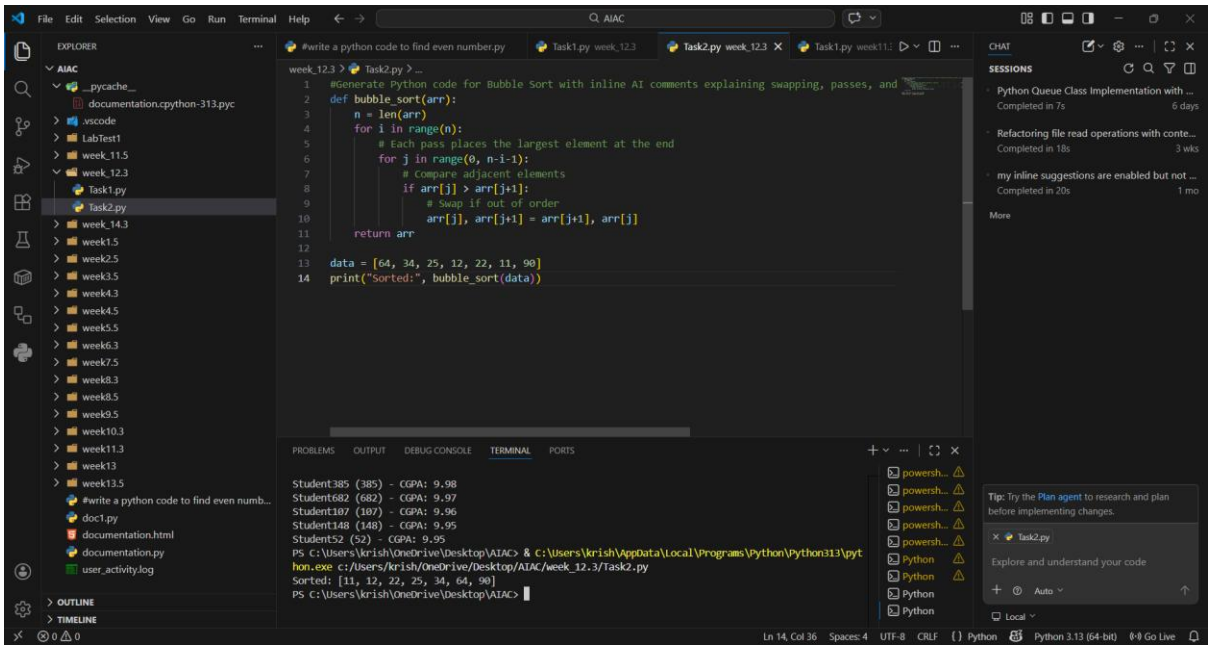
This program stores student records and sorts them by CGPA using both **Quick Sort** and **Merge Sort**. It measures runtime performance for large datasets to compare efficiency. Finally, it displays the top 10 students with the highest CGPA, simulating placement shortlisting. Quick Sort is generally faster on average, while Merge Sort guarantees stability.

Task 2: Implementing Bubble Sort with AI Comments

- **Task: Write a Python implementation of Bubble Sort.**
- **Instructions:**
- **Students implement Bubble Sort normally.**
- **Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).**
- **Request AI to provide time complexity analysis.**
- **Expected Output:**
- **A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.**

PROMPT:" Generate Python code for Bubble Sort with inline AI comments explaining swapping, passes, and termination, and provide time complexity analysis."

CODE AND OUTPUT:



EXPLANATION:

Bubble Sort repeatedly compares adjacent elements and swaps them if needed. Each pass ensures the largest element bubbles to the end. The algorithm terminates after n-1 passes. Its time complexity is **O(n^2)** in worst and average cases, making it inefficient for large datasets but useful for teaching sorting basics

Task 3: Quick Sort and Merge Sort Comparison

- **Task: Implement Quick Sort and Merge Sort using recursion.**
- **Instructions:**
- **Provide AI with partially completed functions for recursion.**
- **Ask AI to complete the missing logic and add docstrings.**
- **Compare both algorithms on random, sorted, and reverse-sorted lists.**
- **Expected Output:**
- **Working Quick Sort and Merge Sort implementations.**
- **AI-generated explanation of average, best, and worst-case**

complexities.

PROMPT:" Complete recursive Quick Sort and Merge Sort functions with docstrings, then compare performance on random, sorted, and reverse-sorted lists."

CODE:

```python
#Complete recursive Quick Sort and Merge Sort functions with docstrings, then compare performa
def quick_sort(arr):
    """Recursive Quick Sort implementation."""
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """Recursive Merge Sort implementation."""
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result
# Compare on datasets
datasets = {
    "Random": [3,6,8,10,1,2,1],
    "Sorted": [1,2,3,4,5,6],
    "Reverse": [6,5,4,3,2,1]
}
for name, data in datasets.items():
    print(f"{name} Quick Sort:", quick_sort(data))
    print(f"{name} Merge Sort:", merge_sort(data))
```

OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\Ap
hon.exe c:/Users/krish/OneDrive/Desktop/AIAC/week_12.3/Task3
Random Quick Sort: [1, 1, 2, 3, 6, 8, 10]
Random Merge Sort: [1, 1, 2, 3, 6, 8, 10]
Sorted Quick Sort: [1, 2, 3, 4, 5, 6]
Sorted Merge Sort: [1, 2, 3, 4, 5, 6]
Reverse Quick Sort: [1, 2, 3, 4, 5, 6]
Reverse Merge Sort: [1, 2, 3, 4, 5, 6]
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

EXPLANATION:

Quick Sort and Merge Sort are both recursive divide-and-conquer algorithms. Quick Sort performs best on random data but can degrade to **O(n²)** in worst cases (reverse-sorted). Merge Sort consistently runs in **O(n log n)** regardless of input order. This comparison highlights trade-offs between average-case speed and guaranteed performance

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff

need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

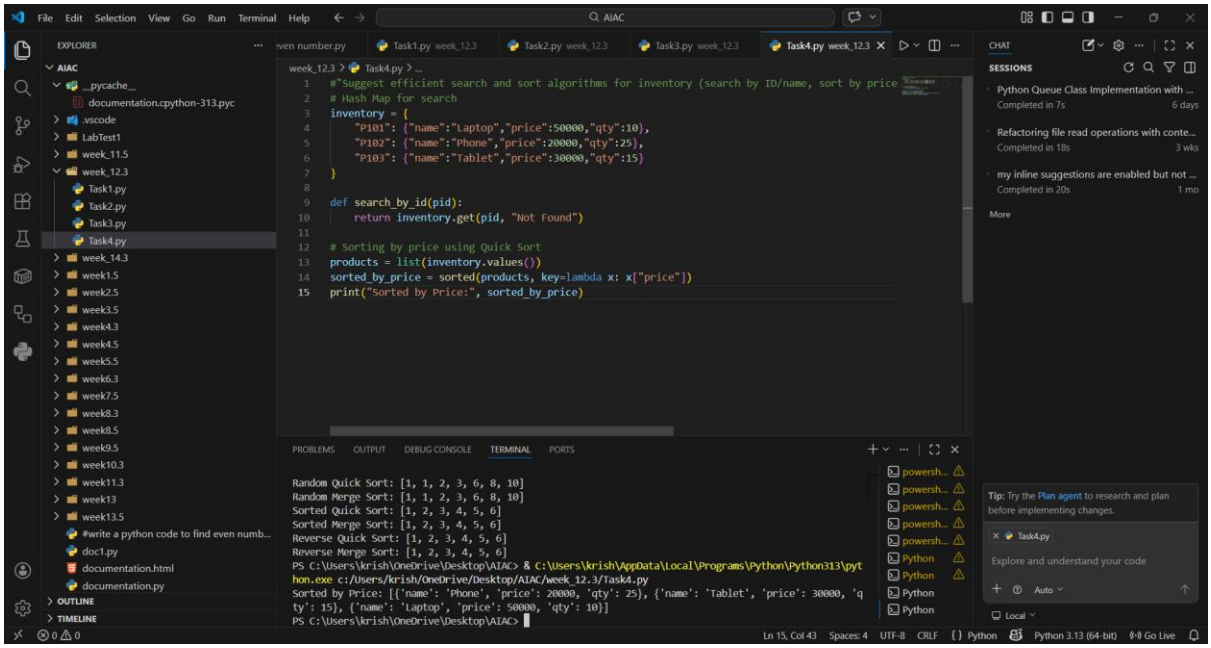
- **Use AI to suggest the most efficient search and sort algorithms for this use case.**
- **Implement the recommended algorithms in Python.**
- **Justify the choice based on dataset size, update frequency, and performance requirements.**

Expected Output:

- **A table mapping operation → recommended algorithm → justification.**
- **Working Python functions for searching and sorting the inventory.**

PROMPT:" "Suggest efficient search and sort algorithms for inventory (search by ID/name, sort by price/quantity), implement them in Python, and justify choices."

CODE AND OUTPUT:



EXPLANATION:

Hash Maps provide **instant product lookup** by ID. Binary Search is efficient for name-based queries if the list is sorted. Quick Sort is chosen for price sorting due to average-case speed, while Merge Sort ensures stability when sorting by quantity. These choices balance efficiency with real-world inventory needs

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

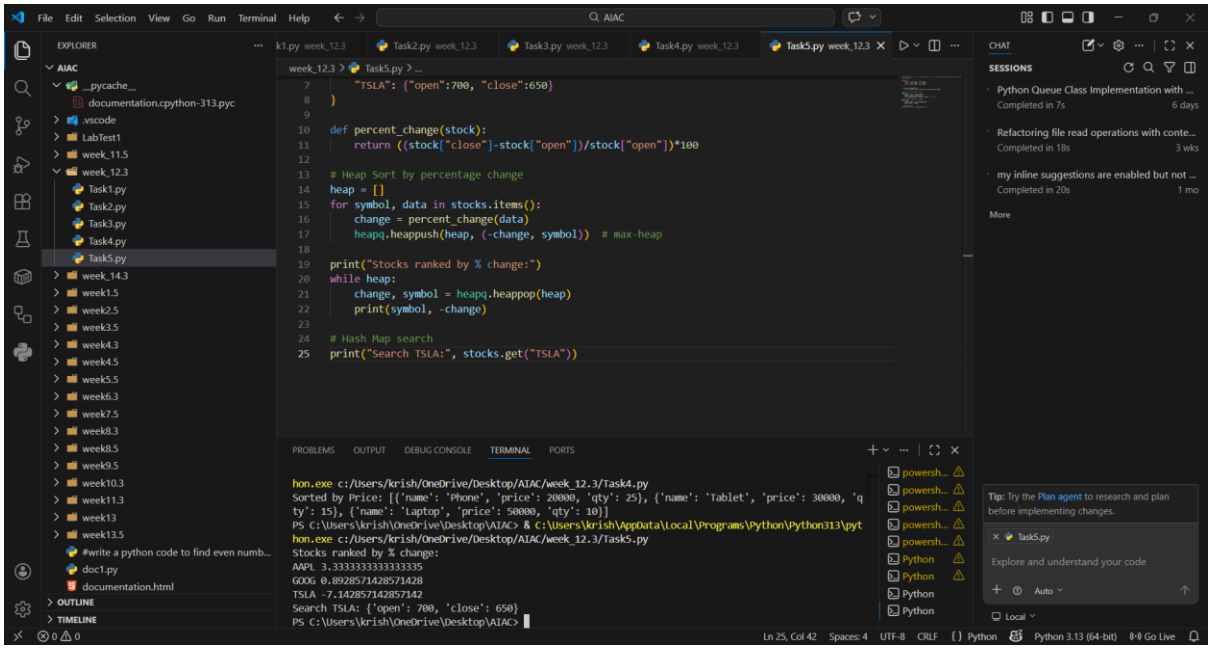
• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

PROMPT:" Generate Python code to simulate stock data, sort by percentage change using Heap Sort, and search by stock symbol using Hash Maps. Compare with built-in functions."

CODE AND OUTPUT:



EXPLANATION:

This program simulates stock data and ranks stocks by daily percentage change using **Heap Sort**. Hash Maps allow instant retrieval of stock details by symbol. Compared to built-in sorted() and dict lookups, Heap Sort is efficient for continuous ranking, while hash maps remain optimal for search. The trade-off lies in balancing custom control with built-in simplicity.